IT Infrastructure Test

PolyWorks Mexico
Candidate: Miguel Angel De La Cruz Espinosa

Instructions: You can answer your test handwriting or digitally, feel free to add screenshots if you need it.

1. Folder Copy and Rotation Script

Write a script in PowerShell to copy the folder `c:/sourcefolder` and its content to this location `d:/destination`. The date and time need to be inserted in the folder's name. The script is going to be executed daily indefinitely but ensure there are always just 10 folders in the destination location.

Output example: `D:/destination/sourcefolder_10202022-1200`

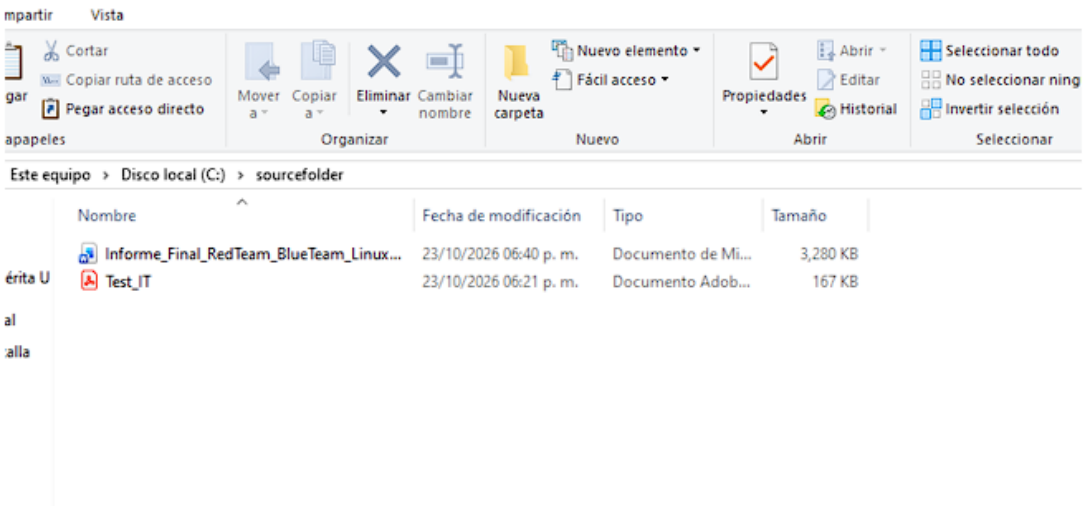


Figure 1: Contents of the source directory (`C:/sourcefolder`) prior to backup execution.

Answer 1: Automated Backup Script

```
<#
.SYNOPSIS
Automated folder backup with a 10-folder retention limit.
#>

$Src = "C:\sourcefolder"
$Dst = "D:\destination"
$TaskName = "AutoBackup-SourceFolder"

# 1. Task Registration
if (!(Get-ScheduledTask -TaskName $TaskName -ErrorAction SilentlyContinue)) {
    $PSArgs = "-WindowStyle Hidden -File '$PSCommandPath'"
```

```
    $Act = New-ScheduledTaskAction -Execute "powershell.exe" -Argument $PSArgs
    $Trg = New-ScheduledTaskTrigger -Daily -At "02:00"

    $TaskParams = @{
        TaskName = $TaskName
        Action   = $Act
        Trigger  = $Trg
        User     = "SYSTEM"
        RunLevel = "Highest"
    }
    Register-ScheduledTask @TaskParams
    Write-Host "[INIT] Task registered successfully." -ForegroundColor Cyan
}

# 2. Execute Backup
$Timestamp = Get-Date -Format 'MMddyyyy-HHmm'
$Tgt = Join-Path $Dst "sourcefolder_$Timestamp"

if (!(Test-Path $Dst)) { New-Item -ItemType Directory -Path $Dst > $null }

if (Test-Path $Src) {
    Copy-Item -Path $Src -Destination $Tgt -Recurse -Force
    Write-Host "[SUCCESS] Backup completed at: $Tgt" -ForegroundColor Green
} else {
    Write-Host "[ERROR] Source folder '$Src' not found." -ForegroundColor Red
    exit
}

# 3. Retention Policy
$OldBackups = Get-ChildItem -Path $Dst -Directory -Filter "sourcefolder_*" |
    Sort-Object CreationTime -Descending |
    Select-Object -Skip 10

if ($OldBackups) {
    $OldBackups | Remove-Item -Recurse -Force
    Write-Host "[CLEANUP] $($OldBackups.Count) backups removed." -ForegroundColor Yellow
} else {
    Write-Host "[INFO] Retention checked. <= 10 folders." -ForegroundColor DarkGray
}
```

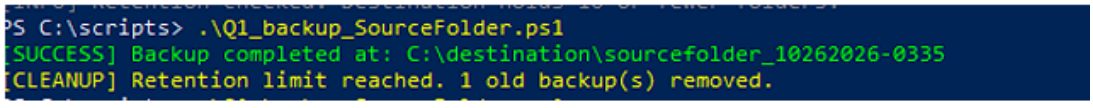


Figure 2: Terminal output showing successful backup execution and old backup cleanup.

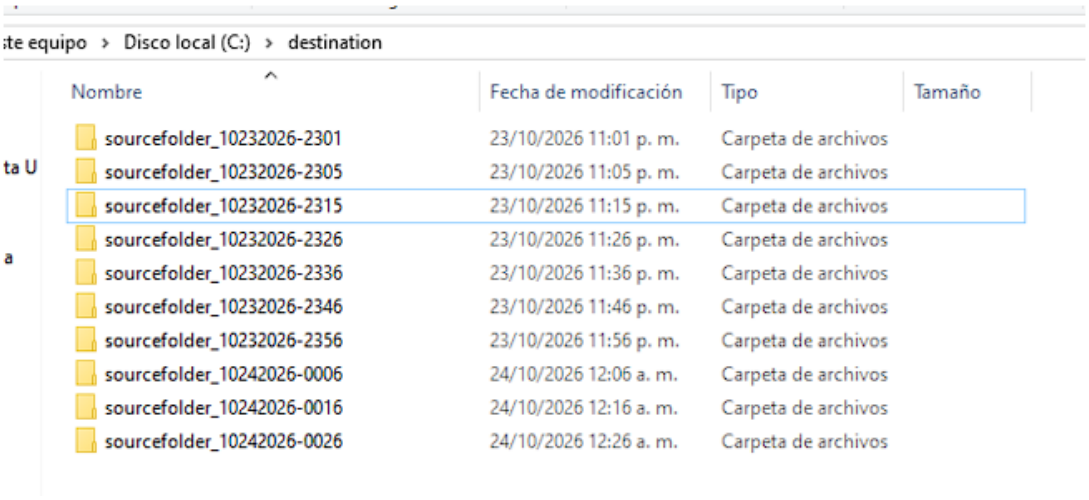


Figure 3: Destination folder view maintaining the 10-folder retention limit.

2. System Information Script

Write a script in PowerShell to get the following system information:

- CPU usage
- Memory usage
- Disk usage

Save each output string into an array and print all the information to a file.
Answer 2: System Information Script

```
# Format timestamp to comply with Windows file naming conventions
$timestamp = Get-Date -Format "dd-MM-yy-HH-mm"
$destination = "C:\temp\SystemInfo${timestamp}.txt"

# Ensure target directory exists
if (-not (Test-Path -Path "C:\temp")) {
    New-Item -ItemType Directory -Path "C:\temp" | Out-Null
}

# 1. CPU Usage
$cpu = (Get-CimInstance -ClassName Win32_Processor |
    Measure-Object -Property LoadPercentage -Average).Average
$cpuString = "CPU usage: $cpu%"

# 2. Memory Usage (CIM returns KB, divide by 1MB to get GB)
$os = Get-CimInstance -ClassName Win32_OperatingSystem
$memTotalGB = [math]::Round($os.TotalVisibleMemorySize / 1MB, 2)
$memFreeGB = [math]::Round($os.FreePhysicalMemory / 1MB, 2)
$memUsedGB = $memTotalGB - $memFreeGB
$memPercentage = [math]::Round(($memUsedGB / $memTotalGB) * 100, 2)

$memString = "Memory usage: $memPercentage% ($memUsedGB GB used out of " +
    "$memTotalGB GB Total | $memFreeGB GB Free)"
```

```
# 3. Disk Usage for C: (CIM returns Bytes, divide by 1GB)
$disk = Get-CimInstance -ClassName Win32_LogicalDisk -Filter "DeviceID='C:'"
$diskTotalGB = [math]::Round($disk.Size / 1GB, 2)
$diskFreeGB = [math]::Round($disk.FreeSpace / 1GB, 2)
$diskUsedGB = $diskTotalGB - $diskFreeGB
$diskPercentage = [math]::Round(($diskUsedGB / $diskTotalGB) * 100, 2)

$diskString = "Disk usage (C:): $diskPercentage% ($diskUsedGB GB used out of " +
    "$diskTotalGB GB Total | $diskFreeGB GB Free)"

# Store strings in an array and export
$sysInfoArray = @(
    "--- System Info Report ---",
    $cpuString,
    $memString,
    $diskString,
    "--------------------------"
)

$sysInfoArray | Out-File -FilePath $destination -Encoding UTF8

$msg = "System metrics successfully exported to: $destination"
Write-Host $msg -ForegroundColor Green
```

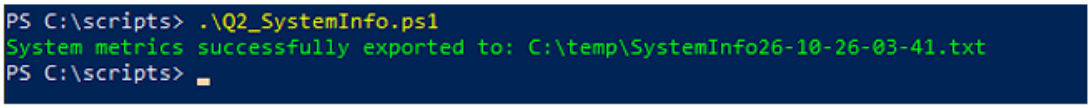


Figure 4: Terminal output confirming the system metrics export.

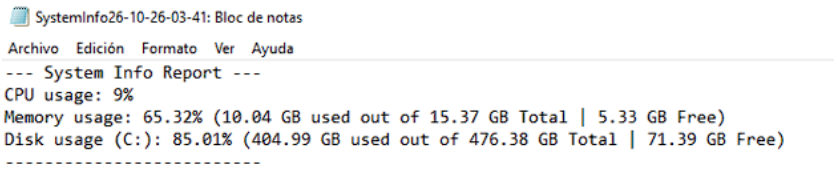
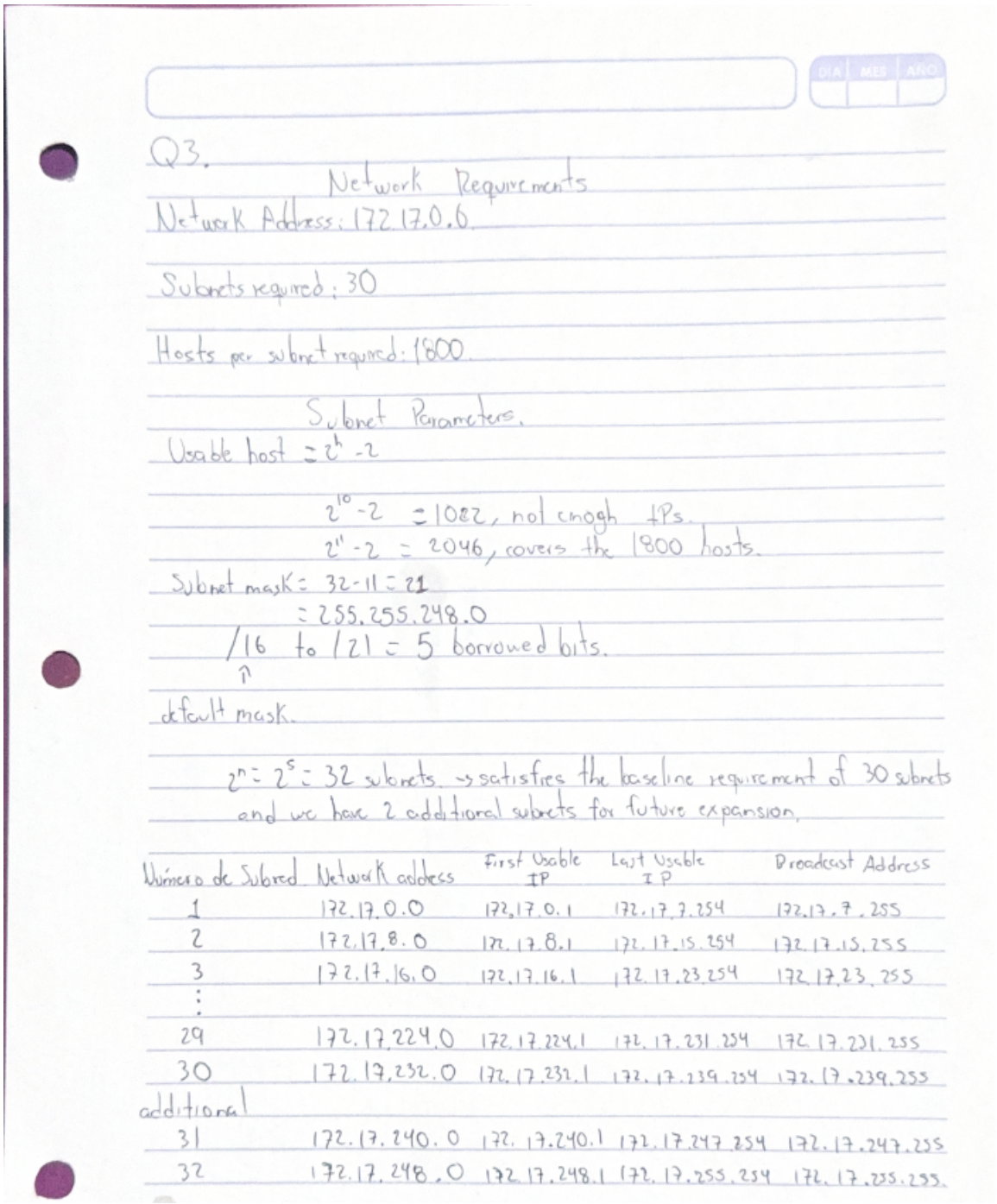


Figure 5: Contents of the generated text file with CPU, Memory, and Disk usage.

3. Subnetting

Given the following network address 172.17.0.0, create 30 subnets and each network must support 1800 hosts. Specify which subnet mask you are going to use and describe why. You can answer handwriting no specific tool or CLI is needed.

Answer 3: Subnetting Calculations



Q3.

Network Requirements

Network Address: 172.17.0.0

Subnets required: 30

Hosts per subnet required: 1800

Subnet Parameters.

Usable host = $2^h - 2$

$2^{10} - 2 = 1022$, not enough IPs.

$2^{11} - 2 = 2046$, covers the 1800 hosts.

Subnet mask = 32 - 11 = 21

= 255.255.248.0

/16 to /21 = 5 borrowed bits.

↑ default mask.

$2^n = 2^5 = 32$ subnets. → satisfies the baseline requirement of 30 subnets and we have 2 additional subnets for future expansion.

Número de Subred	Network address	First Usable IP	Last Usable IP	Broadcast Address
1	172.17.0.0	172.17.0.1	172.17.7.254	172.17.7.255
2	172.17.8.0	172.17.8.1	172.17.15.254	172.17.15.255
3	172.17.16.0	172.17.16.1	172.17.23.254	172.17.23.255
⋮				
29	172.17.224.0	172.17.224.1	172.17.231.254	172.17.231.255
30	172.17.232.0	172.17.232.1	172.17.239.254	172.17.239.255
additional				
31	172.17.240.0	172.17.240.1	172.17.247.254	172.17.247.255
32	172.17.248.0	172.17.248.1	172.17.255.254	172.17.255.255

Figure 6: Handwritten calculations for subnetting 172.17.0.0 for 30 subnets and 1800 hosts.

4. VM Creation and Scheduling

Using PowerShell and commands of any virtualization engine (VMware, Hyper-V, VirtualBox) write a script that executes the following actions:

- Create a virtual machine with the specs below (assume that the OS is going to be installed later)
  - Ram: 8GB
  - CPU: 4cpus
  - Storage: 150GB
  - Location: (default or random location)

Note: You can use random values if needed. The virtual machine is going to be up only during working hours. Write a script and actions to schedule the VM to start at 9:00 am and shut down at 23:00 hrs.

Answer 4: VirtualBox VM Automation Script

```
<#
.SYNOPSIS
    Automates VirtualBox VM creation and task scheduling.
.DESCRIPTION
    INFRASTRUCTURE NOTE: Hardware resources are scaled down to 2GB RAM,
    1 CPU, and 30GB storage due to local host limitations.

    To deploy with the requested specs (8GB RAM, 4 CPUs, 150GB Storage):
    1. modifyvm: Change '--memory 2048' to '--memory 8192'
    2. modifyvm: Change '--cpus 1' to '--cpus 4'
    3. createmedium: Change '--size 30720' to '--size 153600'
#>

$vmName = "PolyWorks_VB_Workstation"
$vboxManage = "C:\Program Files\Oracle\VirtualBox\VBoxManage.exe"
$diskPath = "C:\Users\Public\Documents\$vmName.vdi"

# Create and register the VM
& $vboxManage createvm --name $vmName --register

# Allocate CPU and Memory (Local hardware configuration)
& $vboxManage modifyvm $vmName --memory 2048 --cpus 1

# Create 30GB virtual disk (size in MB)
$diskArgs = @("createmedium", "disk", "--filename", $diskPath,
    "--size", "30720", "--format", "VDI")
& $vboxManage @diskArgs

# Configure SATA controller and attach virtual disk
& $vboxManage storagectl $vmName --name "SATA" --add sata

$attachArgs = @("storageattach", $vmName, "--storagectl", "SATA",
    "--port", "0", "--device", "0", "--type", "hdd", "--medium", $diskPath)
```

```
& $vboxManage @attachArgs

# Schedule background startup at 09:00
$startArgs = "startvm $vmName --type headless"
$startAction = New-ScheduledTaskAction -Execute $vboxManage -Argument $startArgs
$startTrigger = New-ScheduledTaskTrigger -Daily -At "09:00"

$startParams = @{
    TaskName = "Start-$vmName"
    Action   = $startAction
    Trigger  = $startTrigger
    User     = "NT AUTHORITY\SYSTEM"
    RunLevel = "Highest"
}
Register-ScheduledTask @startParams

# Schedule forced shutdown at 23:00
$stopArgs = "controlvm $vmName poweroff"
$stopAction = New-ScheduledTaskAction -Execute $vboxManage -Argument $stopArgs
$stopTrigger = New-ScheduledTaskTrigger -Daily -At "23:00"

$stopParams = @{
    TaskName = "Stop-$vmName"
    Action   = $stopAction
    Trigger  = $stopTrigger
    User     = "NT AUTHORITY\SYSTEM"
    RunLevel = "Highest"
}
Register-ScheduledTask @stopParams
```

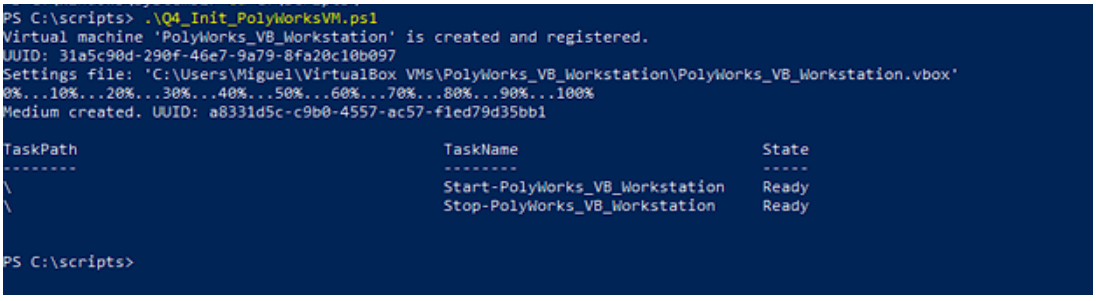


Figure 7: Terminal output confirming VM creation and scheduled task registration.

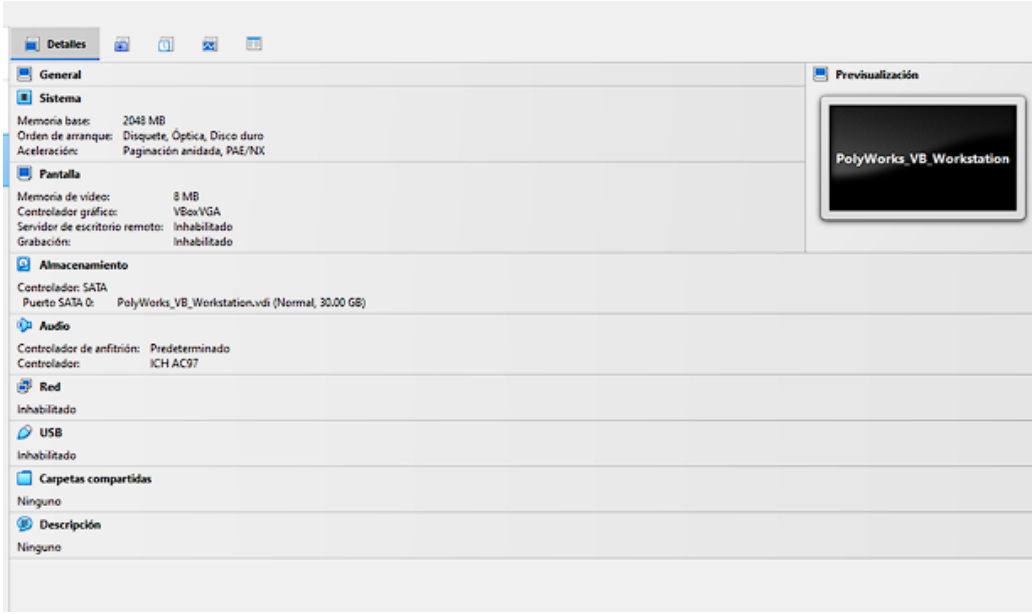


Figure 8: Validation of the newly created Virtual Machine inside VirtualBox Manager.

5. Active Directory Sketch

Sketch a basic structure to deploy an Active Directory for a small company, considering the objects, groups, users, and some departments. It's an open sketch you can use any type of representation for your answer.

Answer 5: Basic AD Structure (Acme Corp)
For a small company (e.g., "Acme Corp").

Legend:

- **DC:** Domain Controller
- **OU:** Organizational Unit
- **SG:** Security Group

```
acme.local (Root Domain)
|
|-- Domain Controllers (Default Container)
|   |-- DC01 (Primary Domain Controller)
|
|-- Computers (Default Container)
|
|-- Acme_Company (Main OU)
    |
    |-- Departments (OU)
    |   |
    |   |-- HR (OU)
    |   |   |-- SG_HR_Staff (Security Group)
    |   |   |-- Alice (User)
    |   |   |-- HR_Desktop01 (Computer)
    |   |
    |   |-- Sales (OU)
    |   |   |-- SG_Sales_Team (Security Group)
    |   |   |-- Bob (User)
    |   |   |-- Charlie (User)
    |   |
    |   |-- IT (OU)
    |       |-- SG_IT_Admins (Security Group)
    |       |-- Dave_Admin (User)
    |
    |-- Shared_Resources (OU)
        |-- Main_FileServer (Computer)
        |-- Network_Printers (SG)
```